

# DocIntel

Offline-First Document Intelligence & Collaboration Platform

Professional Business Requirements, High-Level Design & Low-Level Design

Document status: Architecture baseline / implementation source of truth

| Field           | Value                                      |
|-----------------|--------------------------------------------|
| Project         | DocIntel                                   |
| Version         | 2.0 — Detailed Architecture Baseline       |
| Primary backend | Java 21 + Spring Boot                      |
| Frontend        | React + TypeScript                         |
| Core data       | PostgreSQL + MinIO + OpenSearch            |
| Messaging       | Apache Kafka                               |
| Offline AI      | Local embedding model + optional local LLM |
| Deployment      | Docker Compose first; Kubernetes optional  |

**Purpose of this document: A future reader should be able to understand what the product does, why each architectural component exists, how a document travels through the system, how data is modeled, how APIs and events interact, how failures are handled, and how the system can evolve toward production scale.**

# Document Control

| Item               | Detail                                                                 |
|--------------------|------------------------------------------------------------------------|
| Document name      | DocIntel BRD → HLD → LLD                                               |
| Version            | 2.0                                                                    |
| Scope              | Offline-first enterprise document intelligence platform                |
| Audience           | Developer, reviewer, interviewer, architect                            |
| Baseline principle | Build progressively; preserve this architecture as the source of truth |

## Contents

- 1 BRD — Product & Business Requirements
- 2 HLD — Architecture & System Design
- 3 LLD — Components, schemas, APIs and event contracts
- 4 End-to-end workflows
- 5 Reliability, security, testing and observability
- 6 Implementation roadmap
- 7 SDE-2 engineering/interview surface

## Architecture shorthand

Metadata: PostgreSQL | Binary: MinIO | Async events: Kafka | Search: OpenSearch | Cache: Redis |  
Extraction: Tika | OCR: Tesseract | AI: local embeddings + local LLM

# PART I — BRD

## Business Requirements Document

### 1. Executive Summary

DocIntel is a secure, offline-first platform that converts unstructured enterprise documents into searchable and optionally question-answerable knowledge. A user uploads a document once. The platform stores the original safely, processes it asynchronously, extracts text and metadata, performs OCR when necessary, creates searchable chunks, and optionally generates semantic embeddings. Subsequent lookups use precomputed indexes rather than re-processing the original file.

The project is intentionally designed to demonstrate senior software engineering concerns: asynchronous processing, distributed messaging, data ownership, idempotency, authorization-aware retrieval, versioning, caching, observability, failure recovery and scalable worker execution.

### 2. Business Problem

Organizations commonly have a large number of PDFs, office documents, scans, contracts, specifications and reports. Traditional file repositories make storage and basic lookup possible, but they create friction when users need to locate information inside documents, understand relationships between documents, or answer questions from the document corpus.

DocIntel solves the problem by creating an ingestion-to-knowledge pipeline: file → extracted content → indexed knowledge → authorized retrieval → optional grounded answer.

### 3. Goals

- Reduce time required to locate information inside large document collections.
- Separate durable original-file storage from transactional metadata and search indexes.
- Process expensive document operations asynchronously.
- Provide secure, permission-aware search.
- Provide a clear path from keyword search to semantic search and RAG.
- Remain usable without mandatory cloud APIs.
- Provide enough engineering depth to support SDE-2 system-design discussions.

### 4. Success Metrics

| Metric                    | Target / measurement approach                                                                     |
|---------------------------|---------------------------------------------------------------------------------------------------|
| Upload acceptance         | API accepts a valid file and returns a processing identifier without waiting for full processing. |
| Processing reliability    | Failed jobs retry; permanently failing jobs become visible through a DLQ/recovery path.           |
| Search usefulness         | Relevant documents/chunks appear for representative keyword and semantic queries.                 |
| Authorization correctness | Unauthorized documents never appear in API responses or search results.                           |
| Traceability              | A user can see processing status and an AI answer can reference source document/page/chunk.       |
| Offline capability        | Core stack starts with Docker Compose and does not require cloud AI APIs.                         |

### 5. Actors & Personas

| Actor          | Responsibilities                                        |
|----------------|---------------------------------------------------------|
| Employee       | Upload, search, view, download, comment, ask questions. |
| Document owner | Manage sharing and versions.                            |
| Manager        | Review/share documents and collaborate.                 |

| Actor               | Responsibilities                                                |
|---------------------|-----------------------------------------------------------------|
| Administrator       | Manage users, inspect failures and audit activity.              |
| Processing worker   | Perform extraction/OCR/chunk/index/embedding tasks.             |
| Search/AI subsystem | Retrieve authorized knowledge and construct grounded responses. |

## 6. Functional Requirements

| ID    | Requirement                                                             | Priority |
|-------|-------------------------------------------------------------------------|----------|
| FR-01 | Register/login users and maintain authenticated sessions.               | Must     |
| FR-02 | Upload supported documents and return processing status.                | Must     |
| FR-03 | Store original binaries in object storage and metadata transactionally. | Must     |
| FR-04 | Process documents asynchronously.                                       | Must     |
| FR-05 | Extract text and metadata; route scanned documents through OCR.         | Must     |
| FR-06 | Create page/chunk-level searchable representations.                     | Must     |
| FR-07 | Search authorized documents using keywords, filters and pagination.     | Must     |
| FR-08 | Maintain immutable document versions.                                   | Should   |
| FR-09 | Share documents using viewer/commenter/editor permissions.              | Should   |
| FR-10 | Add page-aware comments.                                                | Should   |
| FR-11 | Generate embeddings and support semantic/hybrid search.                 | Advanced |
| FR-12 | Answer document questions using RAG with citations.                     | Advanced |
| FR-13 | Retry transient processing failures and route terminal failures to DLQ. | Must     |
| FR-14 | Record important audit events.                                          | Should   |
| FR-15 | Expose operational health/metrics.                                      | Must     |

## 7. Non-Functional Requirements

| Category        | Requirement                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------|
| Security        | Server-side authorization; secure password handling; non-public object storage; audit trail. |
| Reliability     | Idempotent consumers, retries, backoff, DLQ, durable source file.                            |
| Scalability     | API and workers can scale independently; search can scale separately.                        |
| Performance     | Interactive search; heavy document processing never blocks upload requests.                  |
| Maintainability | Clear modules, migrations, API contracts, automated tests.                                   |
| Observability   | Logs, metrics and traces connect user request to processing job.                             |
| Offline-first   | No mandatory cloud dependency for the core workflow.                                         |
| Recoverability  | Original files remain available even when indexing/AI dependencies fail.                     |

## 8. User Stories

- As an employee, I want to upload a PDF and immediately know whether it is processing or ready.
- As an employee, I want to search thousands of documents and jump directly to the relevant page.
- As an owner, I want to share a document with different permissions.
- As a user, I want previous document versions preserved.
- As a user, I want to ask a question and see the document/page used to answer it.
- As an administrator, I want failed jobs to be diagnosable and retryable.

## 9. Business Rules

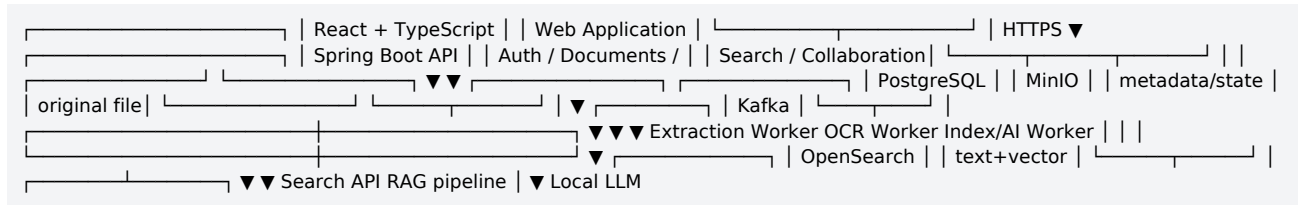
- A document belongs to an owner and can have multiple immutable versions.
- Only authorized users can read metadata, download files, search content or perform collaboration actions.
- A document version is eligible for search only after successful indexing.
- AI retrieval must apply authorization before context reaches the LLM.

- Deleting a logical document must not accidentally delete another document's object-storage data.
- Processing jobs are independent from HTTP request lifetime.

# PART II — HLD

## High-Level Design

### 10. Architectural Overview



### 11. Why Each Component Exists

| Component             | Why it exists                                                        | Source of truth?                           |
|-----------------------|----------------------------------------------------------------------|--------------------------------------------|
| PostgreSQL            | Transactions, relationships, lifecycle, permissions, versions, jobs. | Yes for metadata/state                     |
| MinIO                 | Efficient storage of large binary documents and versions.            | Yes for original files                     |
| Kafka                 | Decouple ingestion from slow/variable processing; absorb bursts.     | Yes for event durability during processing |
| Redis                 | Reduce repeated reads; support rate limits and short-lived state.    | No — cache only                            |
| OpenSearch            | Fast keyword/filter retrieval and optional vector search.            | No — derived index                         |
| Tika                  | Multi-format content/metadata extraction.                            | No                                         |
| Tesseract             | Local OCR for scanned/image-based documents.                         | No                                         |
| Local embedding model | Convert chunks/queries into semantic vectors.                        | No                                         |
| Local LLM             | Generate grounded answers from retrieved chunks.                     | No                                         |

### 12. Logical Boundaries

- API boundary: authentication, authorization, request validation, transactional orchestration.
- Storage boundary: object-storage abstraction prevents MinIO-specific code leaking through the application.
- Processing boundary: workers consume events and produce derived artifacts.
- Search boundary: OpenSearch is treated as a rebuildable projection of authoritative data.
- AI boundary: retrieval is separated from generation so RAG can be evaluated independently.

### 13. Deployment Topology — Local

Docker Compose |— frontend |— backend |— postgres |— redis |— minio |— kafka |— opensearch |— prometheus |— grafana  
Advanced: |— embedding service |— local LLM service |— nginx

## 14. End-to-End Workflow A — First Upload

**Example: user uploads Annual\_Report\_2026.pdf, 80 MB, 250 pages.**

### A1. Client upload

- React selects file and creates upload request.
- Client sends multipart/form-data to POST /api/documents.
- UI may display byte-level upload progress.

### A2. API validation

- Authenticate user.
- Validate MIME type and extension.
- Validate maximum size.
- Optionally calculate/stream checksum.
- Create document/version/job identifiers.

### A3. Transactional metadata

```
documents id = D123 owner_id = U42 status = PROCESSING current_version = 1 document_versions id = V123 document_id = D123
version_no = 1 storage_key = documents/D123/v1/file.pdf processing_jobs id = J789 version_id = V123 status = PENDING
```

### A4. Binary storage

The original file is written to MinIO. PostgreSQL stores the object key, size, checksum and metadata — not the 80 MB binary itself.

### A5. Event publication

```
{ "eventId": "E123", "eventType": "DOCUMENT_UPLOADED", "documentId": "D123", "versionId": "V123", "jobId": "J789", "occurredAt":
  "..."} }
```

### A6. Immediate API response

```
{ "documentId": "D123", "versionId": "V123", "status": "PROCESSING" }
```

The user can close the browser. The processing lifecycle is independent of the HTTP request.

### A7. Worker pipeline

```
Kafka ↓ Fetch file from MinIO ↓ Validate/read ↓ Tika extraction ↓ Text sufficient? └─ yes → continue └─ no → Tesseract OCR ↓
Normalize text + preserve page boundaries ↓ Chunk ↓ Index chunks in OpenSearch ↓ Optional embedding generation ↓ Store vectors
↓ Mark processing READY
```

### A8. Completion

The document is now a reusable knowledge asset. The expensive ingestion work is complete and does not need to be repeated for ordinary lookup.



## 15. End-to-End Workflow B — Second-Time Lookup

**The next day the user searches for Microsoft annual report. There may now be 10,000 documents.**

- 1 React sends GET /api/search?q=Microsoft%20annual%20report.
- 2 Spring Boot authenticates the user.
- 3 Authorization service determines the user's accessible document scope.
- 4 Search service queries OpenSearch with keyword filters; advanced mode may also query vectors.
- 5 Results are ranked and filtered so inaccessible documents are not returned.
- 6 API returns document IDs, titles, snippets, page numbers and scores.
- 7 User selects the annual report.
- 8 Backend retrieves metadata from PostgreSQL and provides controlled access to the MinIO object.
- 9 No Tika/OCR/re-indexing is performed for ordinary lookup.

### Search flow

User query ↓ Authentication ↓ Authorization scope ↓ Search strategy |— Keyword/BM25 |— Vector similarity |— Hybrid ↓ Ranking  
+ permission filtering ↓ Search results ↓ Document metadata / controlled file access

## 16. End-to-End Workflow C — Ask AI

- 1 User asks: 'What was the revenue in 2025?'
- 2 Create query embedding in the advanced mode.
- 3 Run keyword/vector/hybrid retrieval.
- 4 Apply document authorization to retrieved chunks.
- 5 Select top-K relevant chunks.
- 6 Construct a prompt containing the question and retrieved evidence.
- 7 Send only the selected context to the local LLM.
- 8 Return answer plus document/version/page/chunk citations.
- 9 Persist optional feedback/evaluation data.

Critical principle: retrieval is the evidence layer; the LLM is the generation layer. The LLM should not be treated as the source of truth.

## 17. Workflow D — New Version

Existing document D123 v1 ↓ User uploads replacement ↓ Create v2 ↓ Store documents/D123/v2/file.pdf ↓ Publish  
DOCUMENT\_VERSION\_CREATED ↓ Process v2 independently ↓ Index v2 ↓ Set current\_version = 2 v1 remains immutable and recoverable.

# PART III — LLD

## Low-Level Design

### 18. Backend Package Structure

```
com.docintel |— auth | |— controller | |— service | |— domain | |— repository | |— security | |— document | |— controller | |—  
service | |— domain | |— repository | |— mapper | |— storage | |— processing | |— domain | |— service | |— consumer | |—  
publisher | |— search | |— collaboration | |— ai | |— audit | |— common | |— exception | |— validation | |— config | |— observability
```

### 19. Relational Data Model

| Table                | Key columns                                                         | Important constraints                  |
|----------------------|---------------------------------------------------------------------|----------------------------------------|
| users                | id, email, password_hash, status, created_at                        | email unique; status controlled        |
| roles                | id, name                                                            | name unique                            |
| user_roles           | user_id, role_id                                                    | composite unique                       |
| documents            | id, owner_id, name, status, current_version_no                      | owner FK; lifecycle state              |
| document_versions    | id, document_id, version_no, storage_key, checksum, size            | unique(document_id,version_no)         |
| document_permissions | document_id, principal_id, permission                               | unique(document_id,principal_id)       |
| processing_jobs      | id, version_id, status, attempts, next_attempt_at, error_code       | version/job relationship               |
| processing_steps     | id, job_id, step, status, started_at, completed_at                  | unique(job_id,step)                    |
| comments             | id, document_id, version_id, user_id, page_no, text                 | document/user FKs                      |
| audit_events         | id, actor_id, action, resource_type, resource_id, created_at        | append-oriented                        |
| outbox_events        | id, aggregate_type, aggregate_id, event_type, payload, published_at | supports reliable DB→Kafka publication |

### 20. Entity Relationships

```
users 1 — N documents documents 1 — N document_versions documents 1 — N document_permissions document_versions 1 —  
N processing_jobs processing_jobs 1 — N processing_steps documents 1 — N comments users N — N roles users 1 — N  
audit_events outbox_events reference transactional aggregates
```

### 21. Document State Machine

```
UPLOADING → UPLOADED → PROCESSING → READY | ↳ FAILED → RETRY_PENDING → PROCESSING Optional: QUARANTINED DELETED
```

State transitions should be explicit and validated. A worker must not blindly set READY if required processing steps failed.

## 22. REST API Design

| Method | Endpoint                        | Purpose                   | Auth        |
|--------|---------------------------------|---------------------------|-------------|
| POST   | /api/v1/auth/register           | Create user               | Public      |
| POST   | /api/v1/auth/login              | Authenticate              | Public      |
| POST   | /api/v1/documents               | Upload                    | User        |
| GET    | /api/v1/documents               | List authorized documents | User        |
| GET    | /api/v1/documents/{id}          | Get metadata/status       | Permission  |
| GET    | /api/v1/documents/{id}/download | Controlled download/view  | READ        |
| POST   | /api/v1/documents/{id}/versions | Create version            | EDITOR      |
| GET    | /api/v1/documents/{id}/versions | Version history           | READ        |
| POST   | /api/v1/documents/{id}/share    | Grant/revoke access       | OWNER       |
| POST   | /api/v1/documents/{id}/comments | Create comment            | COMMENT     |
| GET    | /api/v1/documents/{id}/comments | List comments             | READ        |
| GET    | /api/v1/search                  | Search                    | User        |
| POST   | /api/v1/documents/{id}/ask      | RAG question              | READ        |
| GET    | /api/v1/jobs/{id}               | Processing status         | Owner/admin |

## 23. Upload API Contract

```
POST /api/v1/documents Content-Type: multipart/form-data Response 202 Accepted { "documentId": "D123", "versionId": "V123", "jobId": "J789", "status": "PROCESSING" }
```

202 is intentional: the server accepted the work; it did not promise that document processing is complete.

## 24. Search API Contract

```
GET /api/v1/search?q=revenue%202025&mode=hybrid&page=0&size=20 { "query": "revenue 2025", "mode": "hybrid", "total": 3, "results": [ { "documentId": "D123", "versionId": "V2", "title": "Annual Report.pdf", "page": 42, "snippet": "...revenue...", "score": 8.71 } ] }
```

## 25. Kafka Event Contracts

| Topic             | Event                    | Purpose                                       |
|-------------------|--------------------------|-----------------------------------------------|
| document.events   | DOCUMENT_UPLOADED        | Start processing of first version             |
| document.events   | DOCUMENT_VERSION_CREATED | Start processing a new version                |
| processing.events | EXTRACTION_COMPLETED     | Signal extracted text is available            |
| processing.events | OCR_COMPLETED            | Signal OCR output is available                |
| processing.events | INDEXING_COMPLETED       | Signal search projection is ready             |
| processing.dlq    | PROCESSING_FAILED        | Terminal failure for manual/recovery handling |

## 26. Event Envelope

```
{ "eventId": "UUID", "eventType": "DOCUMENT_UPLOADED", "schemaVersion": 1, "occurredAt": "ISO-8601", "producer": "document-service", "aggregateId": "D123", "payload": { "documentId": "D123", "versionId": "V123", "jobId": "J789" } }
```

## 27. Processing Worker LLD

```
interface DocumentProcessor { ProcessingResult process(ProcessingContext context); } ProcessingContext: documentId versionId  
jobId storageKey attempt correlationId Pipeline: validate → fetch → extract → OCR? → normalize → chunk → index → embed? →  
complete
```

Each step should have an explicit status. This makes partial failure visible and makes retry behavior easier to reason about.

## 28. Idempotency

Kafka commonly requires consumers to tolerate redelivery. Therefore each processing step uses a deterministic operation key such as versionId + stepName. A unique database constraint on processing\_steps(job\_id, step) or an equivalent idempotency table provides a second line of defense.

Example: if INDEXING\_COMPLETED is published twice, the consumer should produce the same final state rather than duplicate chunks.

## 29. Outbox Pattern

A subtle reliability problem exists between PostgreSQL and Kafka: the application may commit a database transaction and crash before publishing Kafka. To address this in the advanced phase, write the event into an outbox\_events table in the same transaction as the document state change. A publisher later reads the outbox and publishes to Kafka.

```
DB transaction: documents INSERT/UPDATE processing_job INSERT outbox_events INSERT ↓ COMMIT Outbox publisher ↓ Kafka ↓  
Worker
```

This gives a strong interview discussion around at-least-once delivery, atomicity boundaries and eventual consistency.

## 30. Search Design

Index each chunk with document/version/page metadata. Keyword retrieval uses text fields; advanced retrieval stores an embedding vector. Search requests can combine lexical and semantic scores.

```
Chunk ─ documentId ─ versionId ─ chunkId ─ pageStart ─ pageEnd ─ title ─ text ─ permissions / access metadata  
└─ embedding[] (advanced)
```

## 31. Permission-Aware Search

Authorization must be applied before results are exposed. The system should not return ten results and then hide nine in React. The backend constructs an authorization-aware search query or applies a trusted post-filter before serialization. For larger installations, access-control metadata can be denormalized into the search projection and refreshed when permissions change.

## 32. Search Modes

| Mode          | Mechanism                              | Best for                         |
|---------------|----------------------------------------|----------------------------------|
| Keyword       | Text analyzer + BM25/relevance         | Exact terms, identifiers, names  |
| Vector        | Embedding + similarity                 | Conceptual/meaning-based queries |
| Hybrid        | Combine keyword + vector scores        | General-purpose discovery        |
| RAG retrieval | Top-K chunks + authorization + ranking | Question answering               |

### 33. Versioning & Concurrency

Use optimistic locking on mutable document metadata. Each document/version has a version number. A client attempting to update stale state should receive a conflict instead of silently overwriting newer data.

Client reads current\_version = 5 Client A updates → version 6 Client B tries update based on version 5 ↓ CONFLICT ↓ reload / merge / retry

### 34. Collaboration

- Comments reference document/version/page.
- Permissions are explicit and revocable.
- Future real-time collaboration can use WebSockets/SSE, but simultaneous document editing is not required for the core MVP.
- Audit events capture sharing and mutation actions.

### 35. Caching Strategy

| Candidate                | Cache?                     | Reason                                                           |
|--------------------------|----------------------------|------------------------------------------------------------------|
| Document metadata        | Yes, selectively           | High-read/low-change objects.                                    |
| User permission snapshot | Potentially                | Reduces repeated authorization reads; must have invalidation.    |
| Search results           | Short TTL, optional        | Useful for repeated queries; correctness must not rely on cache. |
| Original file            | No Redis                   | MinIO is designed for binary storage.                            |
| Processing state         | Optional short-lived cache | PostgreSQL remains authoritative.                                |

### 36. Rate Limiting

Protect upload, search and AI endpoints independently. For example, uploads are constrained by bytes/requests, search by requests per user, and AI by concurrent generation capacity. Redis can support distributed counters/token buckets in an advanced phase.

### 37. Security Architecture

- Authentication: Spring Security with secure password hashing and token/session strategy.
- Authorization: RBAC plus document-level permissions.
- Object storage: private buckets/objects; access mediated by backend or short-lived controlled URLs.
- Input security: MIME validation, size limits, filename sanitization, decompression/zip-bomb considerations for archive formats.
- Document security: treat uploaded content as untrusted input.
- AI security: retrieved context is bounded by authorization and should be protected against prompt injection from documents.
- Audit: record access and security-sensitive mutations.

### 38. Threat Model — Important Cases

| Threat                           | Mitigation                                                                                  |
|----------------------------------|---------------------------------------------------------------------------------------------|
| Malicious upload                 | Validate type/size; isolate processing; treat parsers/OCR as untrusted boundaries.          |
| Unauthorized search              | Backend authorization filter; never rely on UI.                                             |
| Object URL leakage               | Private storage + short-lived access.                                                       |
| Prompt injection inside document | Treat document text as untrusted data; separate system instructions and retrieved evidence. |
| Duplicate processing             | Idempotency keys + unique constraints.                                                      |
| Event loss                       | Outbox pattern in advanced phase.                                                           |

| Threat                  | Mitigation                                                             |
|-------------------------|------------------------------------------------------------------------|
| Cache stale permissions | Explicit invalidation/versioning; authorization remains authoritative. |

## 39. Testing Strategy

| Level                      | Examples                                                                                |
|----------------------------|-----------------------------------------------------------------------------------------|
| Unit                       | Permission rules, state transitions, chunking, retry classification, ranking utilities. |
| Repository/DB              | Constraints, migrations, optimistic locking.                                            |
| API integration            | Upload, search, sharing, versioning, authorization.                                     |
| Infrastructure integration | PostgreSQL, MinIO, Kafka and OpenSearch via Testcontainers.                             |
| Workflow                   | Upload → Kafka → worker → READY → search.                                               |
| Failure injection          | Worker crash, duplicate event, OCR failure, OpenSearch unavailable, Redis unavailable.  |
| Performance                | Concurrent uploads, processing throughput, search latency, queue lag.                   |
| Security                   | Unauthorized document access, token misuse, malicious input and permission revocation.  |

## 40. Observability Design

Request: React → API traceId=T1 documentId=D123 jobId=J789 Processing: Kafka event → worker traceId=T1 jobId=J789 ↓ Tika ↓ OpenSearch Metrics: API latency upload rate processing duration Kafka consumer lag retry count DLQ count search latency cache hit ratio worker utilization

## 41. Operational Dashboards

- API dashboard: throughput, latency percentiles, error rate.
- Processing dashboard: jobs by state, average duration, retry rate, DLQ size.
- Kafka dashboard: consumer lag and throughput.
- Search dashboard: query latency, failures, index health.
- Storage dashboard: object count and storage growth.
- AI dashboard: retrieval latency, generation latency and evaluation metrics.

## 42. Failure Matrix

| Failure                         | System behavior                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------|
| API process dies during request | Client retries safely using an idempotency key where applicable.                  |
| DB transaction fails            | No document state committed; return error.                                        |
| MinIO write fails               | Do not mark upload complete; clean up partial state if necessary.                 |
| Kafka publish fails             | Outbox records event for later publication in advanced design.                    |
| Worker dies                     | Unacknowledged message can be redelivered; step idempotency prevents duplication. |
| Parser throws                   | Retry if transient; otherwise FAILED/DLQ.                                         |
| OpenSearch unavailable          | Original file remains safe; indexing retries later.                               |
| LLM unavailable                 | Search/view/download continue; AI feature reports unavailable.                    |

## 43. Data Retention & Cleanup

- Keep original versions according to configured retention policy.
- Keep derived search data rebuildable from authoritative content.
- Clean abandoned multipart uploads.
- Define deletion semantics for documents, versions, comments and search projections.
- Audit records may require longer retention than ordinary content.

# PART IV — IMPLEMENTATION ROADMAP

## 44. Progressive Build Plan

| Phase | Deliverable                                   | Key learning                  |
|-------|-----------------------------------------------|-------------------------------|
| 0     | BRD/HLD/LLD + repo standards                  | Requirements and architecture |
| 1     | React + Spring Boot skeleton + Docker Compose | Application boundaries        |
| 2     | Auth + RBAC                                   | Security                      |
| 3     | Upload + PostgreSQL + MinIO                   | Transactions/object storage   |
| 4     | Kafka + one worker + job state                | Async processing              |
| 5     | Tika + OCR + chunking                         | Document processing           |
| 6     | OpenSearch keyword search                     | Indexing/relevance            |
| 7     | Permission-aware search + versions            | Security/concurrency          |
| 8     | Embeddings + vector/hybrid search             | Semantic retrieval            |
| 9     | Local RAG + citations                         | AI systems                    |
| 10    | Retries + DLQ + outbox                        | Distributed reliability       |
| 11    | Metrics + traces + load tests                 | Production engineering        |
| 12    | Kubernetes optional                           | Orchestration/scaling         |

## 45. Definition of Done — Core MVP

- User can register/login.
- User can upload a PDF.
- PDF is stored in MinIO; metadata is in PostgreSQL.
- Kafka event starts processing.
- Worker extracts text and updates status.
- Document reaches READY.
- Keyword search returns authorized results.
- User can open/download the original.
- Automated tests cover the main workflow.
- Docker Compose starts the required local dependencies.

## 46. Advanced Definition of Done

- OCR works for representative scanned PDFs.
- Versioning is immutable and tested under concurrent updates.
- Permission changes affect search visibility.
- Vector/hybrid search works on a representative corpus.
- RAG answers cite source pages/chunks.
- Duplicate Kafka delivery does not duplicate side effects.
- Retry/DLQ behavior is observable.
- OpenTelemetry/Prometheus/Grafana dashboards demonstrate the pipeline.
- Load tests produce measured throughput and bottleneck findings.

## 47. Suggested Repository





# PART V — SDE-2 DESIGN DISCUSSION

## 48. Questions This Project Should Prepare You For

| Topic             | Interview question                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------|
| Storage           | Why MinIO/object storage instead of storing PDFs in PostgreSQL?                               |
| Async             | Why shouldn't upload wait for extraction and OCR?                                             |
| Kafka             | What happens if a worker crashes before acknowledgement?                                      |
| Exactly-once      | Can you guarantee exactly-once processing? What does exactly-once mean at the business level? |
| Outbox            | How do you avoid the DB/Kafka dual-write problem?                                             |
| Search            | Why OpenSearch? Why keyword vs vector vs hybrid?                                              |
| Security          | How do you ensure a user cannot discover an unauthorized document through search?             |
| Scale             | How would you process 10 million documents?                                                   |
| Large files       | How would you handle a 5 GB upload?                                                           |
| Backpressure      | What happens if upload rate exceeds worker processing capacity?                               |
| Versioning        | How do concurrent users avoid overwriting one another?                                        |
| Cache             | What happens if Redis is down or stale?                                                       |
| AI                | How does RAG work and how do you reduce hallucination?                                        |
| AI security       | How would you defend against prompt injection contained in a document?                        |
| Observability     | How would you investigate a sudden increase in processing latency?                            |
| Disaster recovery | What data is authoritative and what can be rebuilt?                                           |

## 49. Architecture Evolution Scenarios

- 1 Start with a modular monolith. Measure actual bottlenecks.
- 2 Split processing workers independently when CPU/OCR load grows.
- 3 Introduce separate worker pools for extraction, OCR and indexing when workloads interfere.
- 4 Add an outbox when reliable DB-to-Kafka publication becomes necessary.
- 5 Add vector search only when keyword search is understood and measurable.
- 6 Add Kubernetes only after containerized local deployment and scaling assumptions are tested.
- 7 Consider multi-tenancy and regional replication only after single-instance behavior is well understood.

## 50. Key Engineering Trade-offs

| Decision               | Trade-off                                                                             |
|------------------------|---------------------------------------------------------------------------------------|
| Async processing       | Higher reliability/scalability; more eventual consistency and operational complexity. |
| OpenSearch projection  | Fast search; index becomes derived state that must be rebuilt/reconciled.             |
| Kafka                  | Durable decoupling; adds operational and conceptual complexity.                       |
| Redis                  | Lower latency; introduces invalidation/staleness concerns.                            |
| Local AI               | Privacy/offline control; higher hardware requirements and model-management work.      |
| Modular monolith first | Simpler development; less independent deployment until modules are split.             |

## 51. Final Source of Truth

Product: DocIntel — Offline-First Document Intelligence & Collaboration Platform.

Core objective: upload once, process asynchronously, index once, then retrieve quickly and securely many times.

Core authoritative stores: PostgreSQL for metadata/state; MinIO for original binaries.

Derived systems: OpenSearch indexes; embeddings; caches.

Core event backbone: Kafka.

Primary engineering themes: asynchronous processing, idempotency, retries, DLQ, authorization-aware search, versioning, observability and scalability.

AI: optional advanced capability using local embeddings and a local LLM; retrieval remains grounded in authorized document chunks.

Build order: core upload → async processing → extraction/OCR → keyword search → security/versioning → semantic search → RAG → reliability hardening → observability → optional Kubernetes.

## **52. First Implementation Milestone**

Implement exactly one vertical slice: register/login → upload PDF → PostgreSQL metadata → MinIO file → Kafka event → worker extraction → READY → keyword search → controlled download. Do not begin with RAG or Kubernetes. Once this path is stable, every later capability can attach to a well-defined document lifecycle.

End of DocIntel Professional Architecture Baseline v2.0